ETHGLOBAL MUMBAI · NOVEMBER 2026

The Idea Book

Four hackathon ideas, expanded into buildable 36-hour plans, each scored out of 10 on how good it is and how likely you are to finish it.

SCORECARD

#	IDEA	ONE-LINER	SCORE
1	**ProofOfRupee**	Prove your income privately with zero-knowledge proofs and get an on-chain credit line, with no credit bureau involved.	8.4
2	**AgentBazaar**	A marketplace where AI agents hire and pay other AI agents per request, using x402 micropayments with escrow.	8.0
3	**Wildcard**	Ball-by-ball cricket prediction markets with gasless, one-tap UX.	7.1
4	**Ghost Ledger**	A trading agent you talk to (Hindi or English) that executes through MEV-protected intents.	6.4

HOW WE SCORED

Each idea gets five sub-scores out of 10, weighted toward **finishing**. A finished 7/10 idea beats an unfinished 10/10 idea at every hackathon.

CRITERION	WEIGHT	WHAT IT MEASURES
Feasibility	30%	Can 3–4 people ship a working, non-faked demo in 36 hours with existing tools?
Wow / novelty	25%	Does it make something possible that wasn't before? Will judges remember it?
Prize stacking	20%	How many sponsor tracks can it enter naturally (ZK, identity, AA, L2s, AI, stablecoins)?
Demo-ability	15%	Can the value be *seen* in a 4-minute live demo?
Local relevance	10%	Will judges and the crowd in Mumbai feel it?

Recommendation: build #1 (ProofOfRupee). It has the highest ceiling and tells the strongest story for India. If your team is stronger in AI and backend than in cryptography, build #2 (AgentBazaar) instead: it is the safer route to the finals.

Before you commit: the Mumbai sponsor list wasn't public when this was written. Check the prize page and map each idea to real tracks. The prize-stacking scores here assume the usual ETHGlobal mix (L2s, ZK and identity, account abstraction, AI agents, stablecoins and payments, oracles). Check that every SDK named here works at the venue in the first 2 hours. If one doesn't, switch early.

Past-event context: Bangkok 2024 finalists covered AI agents, crypto games and DAO tools ($750K pool). Buenos Aires 2025 winners covered NFTs, DeFi and prediction markets (475 teams). Taipei 2025 picked 8 finalists from 226 projects. The pattern in every case: one sharp problem, a live demo that works, and several sponsor tracks combined naturally.

IDEA 1 · RECOMMENDED

ProofOfRupee

8.4/10
Build this

Your bank statement, proven, never revealed. A credit score for India's unbanked, built from zero-knowledge proofs.

THE PROBLEM

Hundreds of millions of Indians earn and spend digitally through UPI but have thin or no credit history, so they can't borrow fairly. DeFi lending requires 150%+ collateral because protocols know nothing about the borrower. Sharing bank statements with lenders is invasive and easy to fake as a PDF.

THE SOLUTION

The user opens their bank or UPI web portal through a **zkTLS** session (Reclaim Protocol or a similar provider). This produces a cryptographic proof that a statement like "*average monthly inflow ≥ ₹50,000 over 6 months*" came from the real bank server, without revealing any transaction. **Anon Aadhaar** adds a ZK proof of "unique Indian resident, 18+" so one person can't farm multiple credit lines. An on-chain verifier mints a non-transferable **credit badge** (income band plus personhood), and a lending pool grants an **undercollateralized USDC credit line** sized to that band.

THE 4-MINUTE DEMO

1. A user with no wallet signs in with email or passkey and gets an embedded smart account (gasless).
2. They scan a test Aadhaar QR and get a "Verified Indian, 18+" proof in seconds.
3. They log in to the bank portal inside the zkTLS flow and get a proof that their income is ≥ ₹50K a month. Show the judges that **no raw data** leaves the device.
4. A credit line of 200 USDC unlocks. They borrow with 20% collateral instead of 150%.
5. A side-by-side "what the lender sees" panel shows only the income band and a nullifier.

ARCHITECTURE

```
[Web app: Next.js + passkey smart account (ERC-4337, paymaster)]
      |-- Anon Aadhaar SDK ----------> proof: age>=18, unique nullifier
      |-- zkTLS SDK (Reclaim etc.) --> proof: income band from bank / UPI page
      v
[CreditVerifier.sol]  verifies both proofs, binds them to one nullifier
      v
[CreditBadge (soulbound ERC-721)]  stores income band, expiry (90 days)
      v
[LendingPool.sol]  credit limit = f(band) | reduced collateral | USDC
      v
[Repayment history]  good repayment raises the band on the next proof
```

TEAM SPLIT (4 PEOPLE)

- **ZK lead:** zkTLS provider setup, custom data provider, proof parsing
- **Contracts:** verifier, badge, lending pool, Foundry tests
- **Frontend:** onboarding, AA wallet, "lender view" panel
- **Product and pitch:** Aadhaar integration, demo script, slides, backup video

LIKELY PRIZE TRACKS

ZK / privacy · Identity · zkTLS provider · Account abstraction · Stablecoins (USDC) · L2 deploy track · Social impact

Expect to qualify for 4–6 tracks without forcing it.

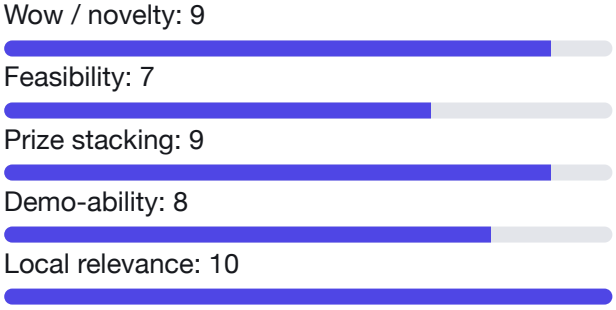
36-HOUR PLAN

Hours 0–4	Hello-world proofs: generate an Anon Aadhaar test proof and one zkTLS proof from any supported site. **Go/no-go checkpoint.**
Hours 4–12	Custom zkTLS provider for the income data source. Contracts: verifier, badge and pool written and tested on a testnet.
Hours 12–22	Wire proofs to contracts end to end. Smart account with gas sponsorship. Basic UI.
Hours 22–30	Lender-view panel, borrow and repay flow, error handling, deploy to the sponsor L2.
Hours 30–36	Polish, record a backup video, 3-slide pitch, rehearse the demo 5+ times.

RISKS AND MITIGATIONS

- **The bank site isn't supported by zkTLS.** Prove from any HTTPS JSON source you can log in to (a payments dashboard, a payroll page), or build a demo bank site. The proof is still real zkTLS.
- **Real Aadhaar data in a demo.** Use the SDK's test mode and test QR codes only.
- **Proof generation is slow on stage.** Pre-generate one proof as a fallback and show the live one in parallel.

SCORE BREAKDOWN



MVP: both proofs → badge → borrow with reduced collateral. **Stretch:** repayment history raises your band; the ZK credit badge is readable by any DeFi protocol (a "credit oracle" interface).

IDEA 2 · SAFEST ROUTE TO THE FINALS

AgentBazaar

The first economy where the workers are AI agents and the wages are paid per request.

8.0/10
Strong alternative

THE PROBLEM

AI agents can't pay each other. Every API needs a human-made account, a credit card and a monthly plan. That blocks the most powerful pattern in AI: one agent hiring a specialist for a single task. **x402** (HTTP 402 Payment Required plus stablecoin settlement) solves the payment part, but there is no marketplace layer: discovery, reputation, or a guarantee that the work was actually done.

THE SOLUTION

A user gives an **orchestrator agent** a goal and a budget, for example: "*Research the top 3 Indian stablecoin projects, write a report, max $3.*" The orchestrator searches an on-chain **agent registry** (capabilities, price, reputation), breaks the goal into subtasks, and pays specialist agents (search, scraping, summarizing, charting, translation to Hindi) **per HTTP call via x402**. Payment for larger jobs sits in **escrow** until a **judge agent** scores the output. Payouts and ratings update on-chain reputation.

THE 4-MINUTE DEMO

1. Type a goal and set a $3 budget.
2. A **live graph** animates: the orchestrator calls 5 agents, each edge flashes with a USDC payment and a block-explorer link.
3. One agent returns poor output. The judge agent rejects it, its escrow refunds, and its reputation drops. The orchestrator re-hires a better agent.
4. The final report appears. Total spend: $1.84, with every cent traceable on-chain.
5. Bonus: an audience member registers their own agent live by deploying an endpoint, and it earns money in the next run.

ARCHITECTURE

```
[User UI: goal + budget | live payment graph (WebSocket)]
      v
[Orchestrator agent (Claude API, tool use)]
      |-- reads --> [AgentRegistry.sol]  capability, price, endpoint, reputation
      |-- pays  --> specialist agents = HTTP endpoints behind x402 paywalls
      |             (search / scrape / summarize / chart / translate)
      |-- big jobs --> [Escrow.sol]  lock USDC -> judge agent score -> release/refund
      v
[Reputation updates on-chain]  + USDC settlement on a cheap L2 testnet
```

TEAM SPLIT (3–4 PEOPLE)

- **Agent lead:** orchestrator planning loop, tool calls, judge agent
- **Payments:** x402 middleware on 5 specialist agents, facilitator setup
- **Contracts:** registry, escrow, reputation
- **Frontend:** live graph (D3 or React Flow), explorer links, pitch

LIKELY PRIZE TRACKS

AI agents | x402 / payments | Stablecoins | L2 deploy track | Agent identity | Storage / data

This is the most popular category, so you need the escrow, judge and reputation loop to stand out.

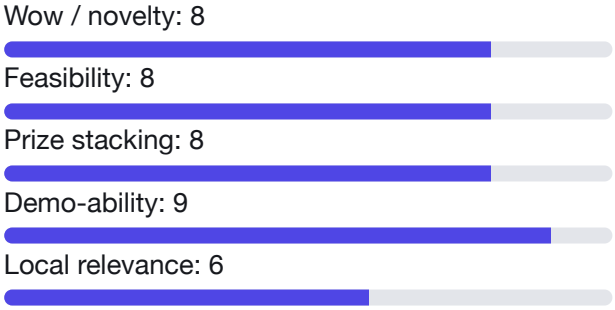
36-HOUR PLAN

Hours 0–4	One x402-protected endpoint plus one client paying it on testnet. **Go/no-go checkpoint.**
Hours 4–14	Five specialist agents behind x402. Registry and escrow contracts.
Hours 14–24	Orchestrator: planning, registry lookup, budget enforcement, judge agent.
Hours 24–32	Live payment graph UI, reputation updates, the deliberate "bad agent" demo path.
Hours 32–36	Polish, backup video, rehearse.

RISKS AND MITIGATIONS

- **Crowded category.** Lead with verification (escrow plus judge plus reputation), not just "agents pay each other."
- **LLM latency on stage.** Use a fast model for specialists and cap the run at about 60 seconds.
- **Facilitator downtime.** Self-host a facilitator or fall back to direct ERC-20 transfers with the same UI.

SCORE BREAKDOWN



MVP: orchestrator plus 3 paid agents plus live graph. **Stretch:** open registration, so anyone can monetize an agent in 1 minute; agents bid on tasks.

IDEA 3 · CROWD-PLEASER

Wildcard

Predict the next over. Ball-by-ball cricket markets that settle in seconds.

7.1/10

Easy to ship, harder to stand out

THE PROBLEM

Cricket is India's biggest live-attention event. Prediction markets are one of crypto's few proven consumer products, but they're built around slow, long-horizon questions. Nobody has made **micro-markets that open and close within a single over**, with UX simple enough for a non-crypto fan.

THE SOLUTION

During a live (or replayed) match, markets open automatically: "*Runs in next over: over or under 7.5?*", "*Wicket this over?*", "*Boundary on the next ball?*". Users join with a phone number or passkey (embedded wallet, gasless) and predict with one tap using **points or testnet tokens**. A **parimutuel pool** settles each market as soon as an **oracle** posts the result. A global leaderboard and shareable "called it" cards drive virality.

THE 4-MINUTE DEMO

1. A judge scans a QR code and is predicting within 10 seconds, with no wallet setup.
2. A replayed match feed advances ball by ball. Markets open and close on screen.
3. The oracle posts the result, payouts land instantly, and the leaderboard updates live for the whole room.

ARCHITECTURE

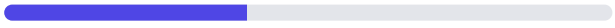
```
[Mobile web app: passkey login, embedded wallet, paymaster]
      v
[MarketFactory.sol]  auto-creates over/ball markets | parimutuel pools
      ^
[Oracle]  cricket data API -> signed result -> on-chain (or optimistic oracle)
      v
[Leaderboard + share cards]  indexer (subgraph) -> UI
```

RISKS AND MITIGATIONS

- **Legal: India banned real-money online gaming (2025).** Use free points or testnet tokens only and pitch it as a skill-and-engagement product. Judges will ask.
- **No reliable on-chain cricket oracle.** Use a signed feed from a sports API, with an optimistic-oracle dispute path as a stretch goal.
- **Low novelty.** Win on UX speed and the live-room leaderboard.

SCORE BREAKDOWN

Wow / novelty: 4



Feasibility: 9



Prize stacking: 6



Demo-ability: 8



Local relevance: 10



36-HOUR PLAN

Hours 0–8	Market and pool contracts, replay feed, signed-result oracle.
Hours 8–20	Mobile UI, passkey wallet, gasless predictions.
Hours 20–30	Leaderboard indexer, share cards, live-room mode.
Hours 30–36	Polish, run a QR-code demo with friends at the venue, rehearse.

Likely tracks: Consumer / UX · Account abstraction · Oracles · L2 deploy track

Make it extraordinary: add a "commentary agent" that explains the odds in Hindi or English, or let fans create their own markets with an AI resolver. This pushes novelty from 4 toward 6.

IDEA 4 · HIGH CEILING, HIGH RISK

Ghost Ledger

"Buy ₹2,000 of ETH if it dips 5%." Say it, and the agent executes it, protected from MEV bots.

6.4/10

Only if you cut the scope hard

THE PROBLEM

Trading on-chain still means slippage settings, approvals, bridges and getting sandwiched by MEV bots. New users, especially non-English speakers, are shut out or lose money to front-running.

THE SOLUTION (SCOPED DOWN FOR 36 HOURS)

A voice or chat agent (Hindi plus English) parses intents like conditional buys, DCA and stop-losses. The agent holds only a **scoped session key** on the user's smart account (spending limits, allowed tokens, expiry), so it *cannot* drain funds. Orders route through an **MEV-protected batch-auction protocol** such as CoW Protocol instead of a public mempool swap. A dashboard shows "MEV saved" against a naive swap.

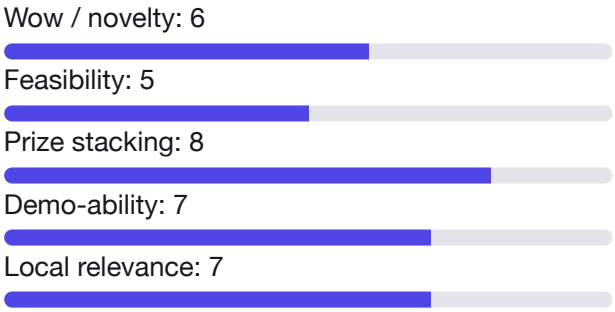
THE 4-MINUTE DEMO

1. Speak an order in Hindi. The agent reads back a plain-language confirmation and the session-key limits.
2. Trigger the condition using a test price feed. The order executes via the batch auction.
3. Show the session key rejecting an over-limit order: the safety story.

RISKS AND MITIGATIONS

- **Too many moving parts** (voice, LLM, session keys, intents, price triggers). Cut cross-chain completely and support only 2 order types.
- **Testnet liquidity is thin.** Check that your intent protocol has a working testnet in hour 1, or demo on a mainnet fork.
- **Crowded "AI trading agent" space.** Lead with the session-key safety story, not the chatbot.
- No financial advice: the agent executes user instructions only.

SCORE BREAKDOWN



36-HOUR PLAN

Hours 0–4	Execute one swap via the intent protocol on testnet or a fork. **Go/no-go checkpoint.**
Hours 4–14	Smart account plus session-key module with limits.
Hours 14–24	LLM intent parser (structured output) plus price-trigger watcher.
Hours 24–36	Voice input, "MEV saved" dashboard, polish, rehearse.

Likely tracks: Intents / DEX · Account abstraction · AI agents · Oracles · L2 deploy track

PLAYBOOK

How to actually win

SIDE-BY-SIDE

IDEA	WOW	FEASIBLE	PRIZES	DEMO	LOCAL	TOTAL
ProofOfRupee	9	7	9	8	10	8.4
AgentBazaar	8	8	8	9	6	8.0
Wildcard	4	9	6	8	10	7.1
Ghost Ledger	6	5	8	7	7	6.4

Total = 0.25 × Wow + 0.30 × Feasibility + 0.20 × Prizes + 0.15 × Demo + 0.10 × Local.

RULES THAT SEPARATE FINALISTS FROM THE REST

1. **Go/no-go at hour 4.** Every plan above starts with its riskiest integration. If it doesn't work by hour 4, switch to your backup idea. Don't sink 20 hours into it.
2. **Talk to sponsor engineers in the first 3 hours.** They tell you what their judges want, and they unblock SDK problems fast.
3. **Pick 3–5 prize tracks, not 10.** Each one should be an integration you can defend in a one-minute explanation.
4. **Demo first, slides second.** Two slides at most (problem, then architecture). Spend the rest of the time on the live product.
5. **Record a backup video** before hour 32. Conference wifi will fail at the worst moment.
6. **Freeze features at hour 30.** Use the last 6 hours for bugs, polish and rehearsing the pitch 5+ times.
7. **Write a strong README** with an architecture diagram, contract addresses and a 2-minute video. Judges read it before you arrive.

SUGGESTED BACKUP PAIRING

Start with **ProofOfRupee**. If zkTLS fails the hour-4 checkpoint, keep the Anon Aadhaar plus lending-pool contracts and switch the income proof to an **AgentBazaar-style** agent that underwrites using paid data agents. Most of your work carries over.

> **Next steps:** (1) get the Mumbai prize list and re-check the prize-stacking scores; (2) assign roles; (3) start the hour-0 proof of concept for your chosen idea.